LogMUSE: Log Anomaly Detection via Multi-Scale Semantic Representation

Mengyao Liu, Tianbo Wang, *Member, IEEE,* Yuan Zhao, Chunhe Xia, Yingming Zeng, and Yuan Tao

***Abstract*—Logs serve as an effective data source for recording and judging system states and abnormal events in complex systems. Current deep learning-based methods have proven effective in detecting anomalies in these system logs. However, existing anomaly detection methods, which predominantly rely on template-based and global window-based approaches, still face challenges in terms of flexibility and practicality. Template-based methods, while widely adopted for their simplicity and efficiency, overlook parameter information and fail to capture the true execution semantics. And global window-based methods, despite their effectiveness in modeling global dependencies, cannot simultaneously capture both global and local dependencies, leading to the obscuration of important local features. To address these issues, we propose a Log anomaly detection method based on MUlti-scale SEmantic representation, LogMUSE. Specifically, LogMUSE obtains template and parameter information through log parsing, employs a pre-trained Bidirectional Encoder Representations from Transformers (BERT) model for template semantic embedding, and enhances log entry representations via cross-attention mechanisms to effectively capture different parameter features under the same template. Additionally, we design the multi-scale Transformer model to capture global and local anomaly patterns, which enable fixed-length log sequences to focus on features at different scales. Extensive experiments on real-world benchmark datasets, including BGL, Thunderbird and Spirit, show that LogMUSE outperforms existing methods in log anomaly detection, achieving F1-scores of 98.62%, 94.32%, and 99.20% respectively. These results surpass the performance of current state-of-the-art methods and demonstrate the strong generalization across different system scenarios.**

***Index Terms*—Log analysis, log parsing, anomaly detection, multi-scale, deep learning.**

I. Introduction

With the rapid advancement of cloud computing, distributed architectures, and microservice technologies, modern computing systems are becoming increasingly complex and large-scale. A typical production environment may consist of thousands of interconnected components deployed across hybrid cloud infrastructures. The increasing dynamism, concurrency, and heterogeneity of these systems introduce unprecedented challenges in system operation, fault diagnosis, and security management.

System logs play a vital role in monitoring system behavior and diagnosing operational issues. They record both high-level user activities and low-level system events, capturing critical information about state transitions, execution flows, and fault symptoms. However, as system scales grow, the volume of log data has expanded dramatically. Public data indicate that Alibaba's production environment can generate over 10 PB of logs daily [1], while major internet companies like Google process billions of events per day [2]. Manual analysis of such massive log data is labor-intensive, error-prone, and incapable of providing timely responses. As a result, log analysis has become a fundamental technique for ensuring system reliability and identifying potential security threats.

Early approaches to log anomaly detection leveraged shallow machine learning models such as Support Vector Machines (SVM) [3], Logistic Regression (LR) [4], and Information-Theoretic Methods (IM) [5], using statistical features extracted from log sequences. However, **these methods struggle to capture complex temporal dependencies and semantic patterns in logs, limiting their detection performance in dynamic environments.**

Subsequent studies introduced deep learning-based models that leverage sequential modeling capabilities. For instance, DeepLog [6], LogAnomaly [7], and LogRobust [8] utilize Recurrent Neural Network (RNN) or Long Short-Term Memory (LSTM) architectures to capture dependencies across log sequences. While effective in certain scenarios, **these models are sensitive to sequence interleaving caused by concurrent tasks**. In practice, system logs are recorded as a single chronological stream, where entries from multiple tasks may be interleaved, breaking the semantic continuity of individual execution paths and impairing sequence modeling.

The Transformer architecture has emerged as a promising alternative for log anomaly detection due to its ability to model long-range dependencies without sequential bottlenecks. LogBERT [9] represents each log message by mapping it to a unique log event template (with parameters removed), and models the resulting sequence of template identifiers as a natural language sequence. Based on this abstraction, it applies the BERT framework with masked token prediction to detect log anomalies. NeuralLog [10] directly tokenizes raw log messages using natural language processing techniques and embeds them with a pre-trained language model. It then applies a Transformer-based classification model for training

Manuscript received 2025. The work was supported by National Engineering Research Center of Classified Protection and Safeguard Technology for Cybersecurity under Grant C24640-XD-04 and the Fundamental Research Funds for the Central Universities. (*Corresponding author: Tianbo Wang*).

M.Y. Liu and T.B. Wang are with the School of Cyber Science and Technology, Beihang University, Beijing, 100191, China. E-mail: {liumengyao, wangtb}@buaa.edu.cn.

Y. Zhao and C.H. Xia are with the School of Computer Science and Engineering, Beihang University, Beijing, China. E-mail: {zhaoyuan_, xch}@buaa.edu.cn.

Y.M. Zeng is with Beijing Institute of Computer Technology and Applications, Beijing, China. E-mail:zengyingming@163.com

Y. Tao is with National Engineering Research Center of Classified Protection and Safeguard Technology for Cybersecurity, the Third Research Institute of Ministry of Public Security, Shanghai, China. E-mail:taoyuan@gass.ac.cn.

and anomaly detection. LogFormer [11] decomposes each log message into a template and its corresponding parameters, and encodes them separately. The parameter embeddings are injected as attention biases, thereby implicitly influencing the representation of the log template. The model is trained using a two-stage strategy comprising pre-training and fine-tuning to perform anomaly detection on log sequences.

While these methods show promise, they still face two major limitations that hinder real-world effectiveness:

First, parameter values, such as IP addresses, file paths, block identifiers, and hardware status codes, often encode context-specific semantics that are crucial for determining the nature of an event. For example, two log messages may share the same parsed template after parameter abstraction, but their parameter values may correspond to different execution contexts, such as different accessed files, affected nodes, or abnormal resource identifiers. Methods that discard parameters or incorporate them only implicitly may therefore assign nearly identical representations to semantically different log instances, making it difficult to detect parameter-sensitive anomalies.

Second, anomalies can manifest across diverse temporal spans. Some anomalies appear as short-term bursts involving only a few adjacent log entries, such as consecutive I/O failures, repeated timeout messages, or machine-check errors within a short sequence. In contrast, other anomalies evolve gradually over longer sequences, such as progressive resource exhaustion, repeated retry-failure patterns, or slowly accumulating hardware error reports. Existing methods often divide log streams into fixed-length windows and apply a uniform modeling scope to the entire sequence. As a result, local abnormal bursts may be smoothed out by global representations, while long-range behavioral drifts may be missed by overly local modeling. Therefore, an effective log anomaly detection model should capture both local and global dependencies.

To address these challenges, we propose a **Log** anomaly detection method based on **MU**lti-scale **SE**mantic representation, **LogMUSE**. Specifically, we explicitly encode both the static template and dynamic parameter components of each log entry, and use a cross-attention mechanism to model their structural dependencies. Furthermore, considering the diversity and abruptness of anomaly patterns in temporal behavior, we introduce a multi-scale windowing mechanism that segments log sequences into subsequences of different granularities. This allows the model to capture both global and local dependencies within the sequence, thereby enhancing its ability to detect various types of anomalies. Experiments on real-world datasets demonstrate that LogMUSE achieves superior performance in log sequence anomaly detection.

The main contributions of this paper are summarized as follows:

1) We propose a novel semantic modeling strategy that explicitly aligns parameters with templates through a cross-attention mechanism. This design enables fine-grained, instance-specific log representations, enhancing robustness to parameter-sensitive anomalies. In this way, our approach effectively addresses the challenge of semantic degradation caused by template-parameter abstraction.

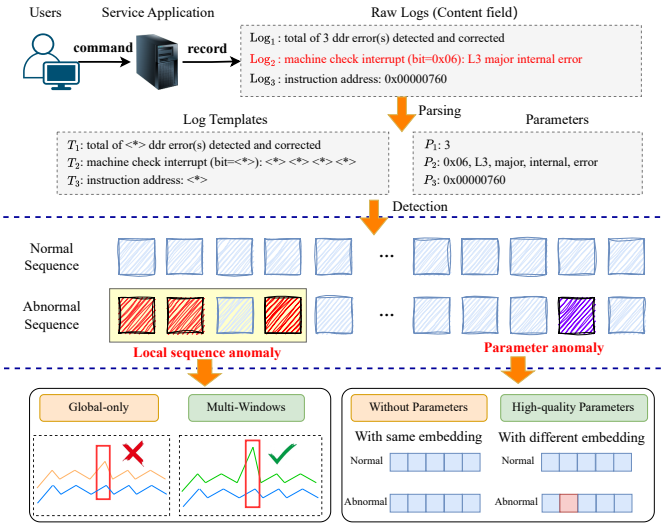


Fig. 1: Illustration of the real-world log processing workflow and key challenges in anomaly detection

2) We design a multi-scale attention-based Transformer architecture that integrates representations across different window scales. This architecture captures both global and local dependencies in log sequences, thereby improving the model's sensitivity to diverse anomaly patterns. Consequently, it addresses the challenge of temporal heterogeneity in anomaly manifestation.
3) We conduct a comprehensive empirical evaluation on three real-world datasets, including BGL, Thunderbird, and Spirit. The results demonstrate that LogMUSE outperforms representative state-of-the-art baselines, validating the effectiveness and generalizability of our method.

The remainder of this paper is organized as follows: Section II introduces the background of log parsing and the model assumptions. Section III presents the details of LogMUSE. Section IV reports the experimental evaluation. Section V discusses the effects, limitations, and future work. Section VI reviews related work, and Section VII concludes this paper.

II. Background and Motivations

A. System Logs and Log Parsing

As illustrated in Figure 1, log entries are typically produced in response to user commands or internal system triggers, capturing diverse runtime events such as memory errors, processor interrupts, or I/O activities.

These raw logs are unstructured, with the core semantic content embedded in the **Content** field, usually a free-text description of system events. To facilitate downstream analysis, **log parsing** is applied to convert unstructured log messages into structured forms. As shown in Figure 1, log parsing replaces dynamic parameters (i.e., variable tokens such as IP addresses, file names, or numeric values) with wildcards (e.g., <*>), while preserving the constant parts of the message. The resulting sequence, composed of static tokens and wildcards, is referred to as a "log keys" or "log

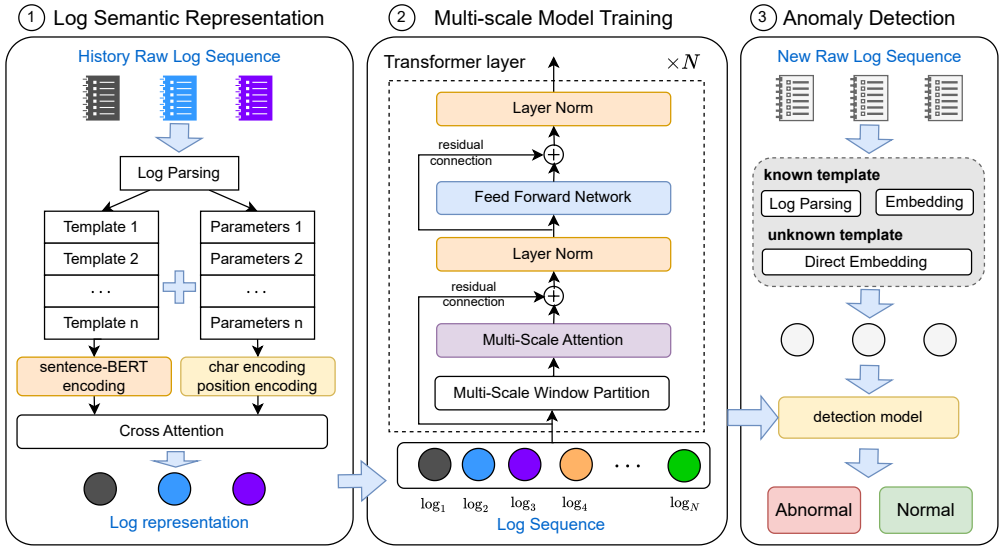


Fig. 2: Architecture of LogMUSE

template", which serves as a structured representation of the original message for downstream modeling. Numerous parsing methods, such as Drain [12], Spell [13], MoLFI [14], IPLoM [15], and LLMParser [16] have been proposed to automate and optimize this process. In anomaly detection scenarios, log parsing is widely adopted to enable sequence learning based on log templates [6], [7], [9].

B. Challenges

While log parsing helps extract structured representations from unstructured messages, it also introduces new challenges when building effective representations for anomaly detection, especially in the presence of complex semantics and temporal dynamics, as shown in Figure 1.

Semantic degradation from template-parameter abstraction. Log parsing separates log content into templates and parameters, where parameters are abstracted as wildcards (e.g., <*>). This abstraction enables structural generalization but discards rich semantic information embedded in parameter values, such as IP addresses or process IDs. Moreover, the alignment between template tokens and their corresponding parameters is often lost, resulting in incomplete semantics for downstream modeling. Since many anomalies arise from subtle shifts in parameter usage rather than structural changes, such degradation hampers the accurate detection of parameter-sensitive anomalies.

Temporal heterogeneity of anomalies. Anomalies may emerge at different temporal resolutions - some as sudden bursts, or others as slow behavioral drifts. However, many existing methods model log sequences using fixed-size windows and global representations, which tend to smooth out local variations or overlook long-term correlations. This uniform modeling limits their ability to detect diverse temporal patterns.

C. Model Assumption

In this study, we assume that the log data used for training and testing is complete, authentic, and free from tampering. Specifically, we assume that the log data has not been subjected to any form of attack (e.g., tampering, poisoning, or deletion) during collection, storage, or transmission, and that the log entries accurately reflect the true state and behavior of the system. Therefore, the abnormal patterns we can detect include:

- Abnormal behaviors caused by improper or erroneous user operations during system runtime. These behaviors may deviate from the system's normal operational norms, leading to discrepancies between the system state or execution flow and the expected outcomes, thereby leaving anomalous information in the logs.
- Abnormal system executions caused by attacks. These attacks can compromise system integrity, bypass security mechanisms, or exploit system resources, leading to significant deviations from expected behavioral patterns and resulting in anomalous log patterns.

III. Design of LogMUSE

A. Overview

To effectively detect anomaly log sequences, we propose LogMUSE, a novel log anomaly detection method based on log semantics representation and multi-scale feature training. The framework of LogMUSE, as shown in Figure 2, consists of three main components: log semantic representation, Multi-scale model training, and anomaly detection.

The log sequence is used as input. For each log, we first extract the template and corresponding parameter information. Then, we preprocess the template and parameter data separately to obtain their semantic embeddings. In anomaly detection, our goal is to detect both sequence order anomalies and parameter value anomalies. Therefore, we handle the semantic

TABLE I: Notations used in LogMUSE

Symbol	Definition
S	A single raw log entry
X^E	Log template embedding
P^E	Parameter embedding
X^E_{final}	Log entry embedding
Seq	Log sequence
d	Embedding dimension
N	Length of log sequence
ω	Attention radius
Ψ	Set of multi-scale attention radii
N'	The number of different attention scales
$Attn_{fused}(Seq)$	Multi-scale attention output for the sequence
Z_{output}	Logits vector for binary classification

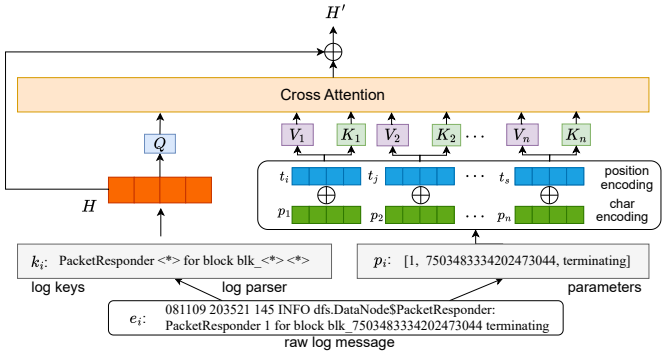


Fig. 3: Detailed view of log message embedding using Cross-Attention

embeddings of log templates and parameter embeddings separately to achieve a complete log semantic representation. Since the focus of our work is log anomaly detection rather than log parsing, we utilize the state-of-the-art log parser, Drain [12], to extract the templates and corresponding parameter information from the logs.

For the log templates, we directly use a pre-trained BERT model to obtain their semantic information, while the parameter embeddings are obtained through a joint embedding of characters and positions. After acquiring the semantic embeddings of the log templates and parameter embeddings, we design a cross-attention mechanism to enhance the semantic representation of the log templates using log parameters. This approach allows the model to increase its sensitivity to parameter features without deviating from the sequence semantics.

Finally, we propose a log sequence anomaly detection model based on multi-scale features, where different scale-specific attention ranges are used within the Transformer attention mechanism. The motivation behind this design is that anomalies in log sequences are not always determined by the entire sequence; by using attention ranges of varying scales, we can capture different dependency relationships more effectively. Relying solely on global sequence features may smooth out the overall patterns, making it difficult to extract local features. In contrast, smaller scales enhance local sharpness, thereby improving detection performance. Table I summarizes the main notations employed in this work.

B. Log Semantic Representation

Log entries consist of static templates and dynamic parameters, capturing fixed execution patterns and runtime-specific contexts, respectively. Effectively modeling their combined semantics is crucial for accurate anomaly detection. However, existing methods often underutilize parameter information, limiting their ability to detect anomalies driven by parameter variations. To address this, we design a semantic representation module that explicitly incorporates parameter content and position into the template embedding, enabling instance-specific log representations.

In log anomaly detection, abnormal behavior may manifest as point anomalies and sequential deviations. Given a log entry S, our objective is to generate an embedding that preserves the structural semantics of the template while distinguishing variations arising from parameter content. Figure 3 illustrates the semantic representation process: the parameter and template representations are obtained individually and then fused using a cross-attention mechanism to produce the final log representation.

Specifically, we first perform log parsing to separate the template and its corresponding parameters. The template can be represented as $X = \{x_1, x_2, \cdots, x_n\}$, where x_i is the i-th token, and n represents the length of the log template. Here, we use the pre-trained sentence-BERT [17] to get the template embedding, which is chosen for its capability to generate semantically meaningful sentence-level embeddings while preserving computational efficiency. The embedded log template vector can be expressed as:

$$X^E = f(X) \tag{1}$$

where $X^E \in \mathbb{R}^{1\times d}$, and $f(\cdot)$ is the encoder of sentence-BERT.

The log template embedding captures the constant structure of the log messages, offering rich semantic representation and addressing issues related to unseen log templates. However, relying solely on template embeddings overlooks the dynamic information contained in parameters, making it challenging to detect anomalies arising from abnormal parameter changes. Therefore, we separately encode the parameters and integrate them with the template embedding to construct a complete semantic representation.

Given a log entry, we extract its parameter list $Param = \{param_1, param_2, \cdots, param_m\}$ through log parsing, where m represents the number of parameters. Since parameters are often variable-length strings (e.g., file paths, IP addresses), we represent each parameter as a sequence of character embeddings and apply mean-pooling to obtain a fixed-dimensional vector $P_j = C(param_i), P_j \in \mathbb{R}^{1\times d}$, and the encoding of each parameter has the same dimension d as the log template embedding.

However, parameters are encoded independently of the log template, and relying solely on their relative order in the parameter list fails to capture their true contextual roles. For example, whether a token represents a *user ID* or a *host ID* depends on its absolute position within the original log message, rather than its index among parameters. Therefore,

we introduce absolute positional encoding based on each parameter's starting character offset in the raw log text.

Following the sinusoidal encoding scheme in [18], we define the positional encoding $T \in \mathbb{R}^{1\times d}$. The encoding can be defined as: $T_{t,2l} = sin(t/10000^{2l/d})$, $T_{t,2l+1} = cos(t/10000^{(2l+1)/d})$, where t represents the position of the word in the original vector, and l represents the encoding of the l-th dimension position. For a given parameter $param_i$, we locate its start index in the original log string using $x = S.index(param_i)$. The absolute position embedding vector for this parameter is then $T_x \in \mathbb{R}^{1\times d}$, i.e., the entire vector at position x. We combine this position vector with the parameter's content embedding:

$$P_i^E = P_i + T_{x=S.index(param_i)} \tag{2}$$

where P_i^E represents the embedding of the i-th parameter. Then the encoding of all parameters can be expressed as $P_{seq}^E = [P_1^E, P_2^E, \cdots, P_m^E]$.

To effectively integrate the semantic information of the log template and its parameters, we design a multi-head cross-attention mechanism, where the log template embedding X^E serves as the query source, and the parameter embeddings P_{seq}^E serve as the key and value sources. Specifically, for each attention head, we compute the attention-enhanced representation of the log template as follows:

$$Attention^{(h)} = softmax\left(\frac{QK_{seq}^T}{\sqrt{d/h}}\right)V_{seq} \tag{3}$$

where the query $Q = X^E W_Q^{(h)} + b_Q^{(h)}$ is derived from the log template, and the key and value are derived from the parameter sequence: $K_{seq} = P_{seq}^E W_K^{(h)} + b_K^{(h)}$, $V_{seq} = P_{seq}^E W_V^{(h)} + b_V^{(h)}$.

This design allows the log template to selectively attend to different parts of the parameter sequence, enriching its semantic representation with contextually relevant information from the parameters.

Multi-head attention uses a parallel combination of multiple attention mechanisms to capture different aspects of information between log templates and parameters. We connect H heads of multi-head attention and map them to the final template enhancement vector $X_{context}^E$ through a linear layer:

$$X_{context}^E = Concat(Attention^{(1)}, \cdots, Attention^{(H)})W_O \tag{4}$$

Finally, the original template semantic embedding is superimposed with the enhanced template representation to obtain the final log semantic representation:

$$X_{final}^E = X^E + X_{context}^E \tag{5}$$

C. Multi-scale Model Training

While modeling individual log entries effectively captures deep semantic features, robust log anomaly detection also requires modeling the overall semantic patterns across sequences of logs to better understand system behaviors. Traditional methods typically perform unified modeling over the entire sequence within a fixed-size window, aiming to capture anomaly features at a global level. However, in real-world systems, anomalies often exhibit diverse temporal patterns: some manifest as local bursts (e.g., sudden surges in error events), while others develop gradually over extended periods (e.g., progressive resource exhaustion). This diversity poses a significant challenge, as global modeling tends to smooth out localized anomalies, reducing detection sensitivity and accuracy.

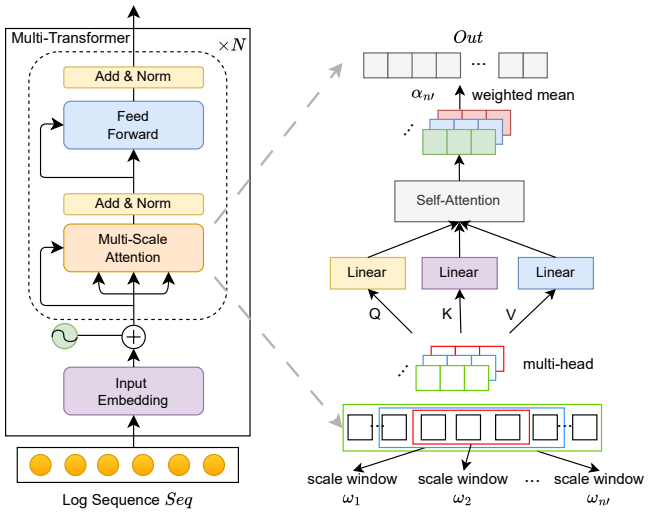


Fig. 4: Log sequence embedding with Multi-Scale Transformer blocks

To address these challenges, we propose a multi-scale Transformer-based framework for log sequence anomaly detection, as shown in Figure 4. This framework explicitly models log subsequences at multiple granularities in parallel, enabling the detection of both short-term bursts and long-range dependencies. While previous Transformer-based methods [10], [11], [19]–[21] primarily model global dependencies, our design extends the standard Transformer architecture by incorporating multiple scale-aware attention mechanisms operating at different temporal scales.

Specifically, given a log sequence $Seq = \{X_{final_1}^E, X_{final_2}^E, \cdots, X_{final_N}^E\} \in \mathbb{R}^{N\times d}$, where N denotes the sequence length and d denotes the embedding dimension, we introduce a set of scale parameters $\Psi = \{\omega_1, \omega_2, \cdots, \omega_{N'}\}$ to define different attention radii for scale-aware self-attention. Each $\omega_{n'}$ controls the receptive field around the current log entry. A larger $\omega_{n'}$ allows each log entry to attend to a broader contextual range and is suitable for capturing long-range behavioral dependencies, while a smaller $\omega_{n'}$ restricts the attention range to neighboring entries and emphasizes relatively local deviations.

We adopt dyadic scales (i.e., $\Psi = \{N/(2^k)\}$ for relevant k) as candidates. This choice follows the principle of multiresolution analysis [22], where powers of two provide a natural logarithmic coverage of the scale space, ensuring that dependencies ranging from short-term bursts to long-term drifts are captured efficiently. Moreover, dyadic scaling achieves higher computational efficiency and implementation simplicity compared with arbitrary partitions, while preserving

complementary information across scales.

At each scale $\omega_{n'}$, the entire sequence is processed using a dedicated scale-aware self-attention module. For a log entry at position n, the attention range is restricted to $\mathcal{W}_n = [max(0, n-\omega_{n'}), min(N, n+\omega_{n'})]$. Specifically, when $\omega_{n'} = N$, the attention range covers the entire sequence, recovering standard Transformer behavior. Within this scale-specific attention range, the multi-head attention scores are computed as:

$$Attn_{\omega_{n'}}(Seq)_n = softmax(\frac{Q_{n,i}K^T_{\mathcal{W}_n,i}}{\sqrt{d_k}})V_{\mathcal{W}_n,i} \quad (6)$$

where i represents the i-th attention head, $Q = SeqW_Q$, $K = SeqW_K$, $V = SeqW_V$, and $Q_{n,i}$ is the query vector at position n, while $K_{\mathcal{W}_n,i}$, $V_{\mathcal{W}_n,i}$ are the key and value vectors within the attention range $\mathcal{W}_n$. This scale-aware attention mechanism enables the model to focus on different temporal ranges of the sequence, capturing localized dependencies under smaller radii while preserving broader contextual information under larger radii, thereby alleviating the issue of feature smoothing in long sequences.

To integrate information across all temporal scales, we perform multi-scale fusion. Each scale $\omega_{n'}$ produces an attention output $Attn_{\omega_{n'}}(Seq)$ for the entire sequence. The outputs from all N' scales are then combined through a learnable weighted summation:

$$Attn_{fused}(Seq) = \sum_{n'=1}^{N'} \alpha_{n'} \cdot Attn_{\omega_{n'}}(Seq) \quad (7)$$

where $\alpha = \mathrm{softmax}([\alpha_1, \cdots, \alpha_{N'}])$ are learnable parameters that determine the relative importance of each temporal scale for anomaly detection. It should be noted that these fusion weights are learned globally over the training distribution, rather than dynamically generated for each individual sample. Nevertheless, per-instance adaptivity is still implicitly achieved through the multi-head attention mechanism within each scale, where each log entry attends to different contextual entries according to its own query representation.

We replace the standard self-attention layer in the Transformer with our multi-scale attention mechanism to form the Multi-Scale Transformer layer. The complete layer follows the standard Transformer architecture with residual connections and layer normalization:

$$X_{attn} = LayerNorm(X^E_{final_n} + Attn_{fused}(X^E_{final_n})) \quad (8)$$

$$X_{out} = LayerNorm(X_{attn} + FFN(X_{attn})) \quad (9)$$

Each sequence embedding is augmented with sinusoidal positional embeddings to retain sequential information. After passing through the stacked Multi-Scale Transformer layers, we obtain the output sequence representation. To produce the final classification, we apply global average pooling across the sequence dimension to aggregate features from all time steps into a single sequence vector:

$$h_{trans} = \frac{1}{N}\sum_{n=1}^{N} Trans(Seq)_n \quad (10)$$

where h_{trans} denotes the sequence representation after global average pooling. This aggregated representation is then fed into a fully connected layer for binary classification:

$$Z_{output} = W_{fc} \times h^T_{trans} + b_{fc} \quad (11)$$

where Z_{output} is a 2-dimensional vector representing the final classification result, distinguishing between normal and anomalous log sequences.

Finally, during training, we compute the cross-entropy loss between the predicted results and the ground truth labels to optimize the model, which in turn helps adjust the model parameters for the final classification task.

D. Anomaly Detection

Through the aforementioned steps, we can train a multi-scale Transformer-based model for log anomaly detection. When a new set of log sequences arrives, each log message is first preprocessed to extract its log template. The extracted template is then embedded using sentence-BERT, and its representation is further enhanced by incorporating parameter semantics. For unseen or rare templates that cannot be matched by the parser, we directly embed the entire log message using sentence-BERT, without applying template-parameter separation:

$$X^E_{oov} = f(Seq_{unknown}) \quad (12)$$

This design ensures robustness in real-world deployments, where new log templates frequently emerge. Although these embeddings may lack the fine-grained parameter enhancement, they still provide a semantic representation consistent with the overall embedding space, thus allowing the model to handle unseen patterns without retraining.

Subsequently, the trained model is employed to learn the sequence features of the log sequence, enabling the determination of whether the sequence is normal or anomalous.

$$result = softmax(Z_{output}) \quad (13)$$

IV. Evaluation

In this section, to discuss the effectiveness and advantages of our proposed LogMUSE in detail, we conduct a series of experiments to evaluate it. Specifically, the **R**esearch **Q**uestions (**RQs**) below guided the experiments we evaluated.

- **RQ1**: How effective is LogMUSE in detecting anomaly log sequences in comparison with other state-of-the-art techniques?
- **RQ2**: How does LogMUSE perform in terms of log semantic embedding?
- **RQ3**: What is the impact of multi-scales on LogMUSE's performance?
- **RQ4**: Which model configuration leads to the best performance of LogMUSE?

A. Experiment Setup

Datasets. We evaluate the proposed LogMUSE on the three most commonly-used real-world datasets [23], BGL, Thunderbird, and Spirit. The BGL dataset is collected from the BlueGene/L supercomputer system at Lawrence Livermore National Labs (LLNL). This dataset contains 4,747,936 log messages labeled as anomalous or normal. The Thunderbird dataset is collected from a real-world supercomputer system at Sandia National Labs (SNL), with the log data labeled as anomalous or non-anomalous. In this experiment, we leverage 10 million continuous log messages from the Thunderbird dataset, as previous work did [10], [11]. The Spirit dataset is collected from the Spirit supercomputer system at LLNL, spanning a period of 2.5 years. This dataset contains a total of 272,298,969 log messages, with labels for anomalous or normal entries [24]. In these three datasets, since there is no predefined target partitioning criterion, we adopt the same partitioning method as in previous work [6], [25], using a window size of 20 for data segmentation. We further analyzed the effect of this setting on anomaly composition. For the BGL dataset, a window size of 20 contains at most four anomaly types, and only 18 windows contain more than two anomaly types. Larger windows, such as a fixed size of 50 or time-based windows, increase the number of multi-type anomaly windows to 37 and 138, respectively, which may dilute anomaly signals and make precise localization more difficult. This issue may also occur on Thunderbird and Spirit, since larger windows are more likely to mix heterogeneous abnormal events within the same sequence. Thus, a fixed window size of 20 provides a practical balance for the tasks in this study. Finally, for all log sequences, the first 80% of the data is used for training, while the remaining 20% is used for testing.

Baselines. We compare LogMUSE with seven representative baseline methods. We select these methods based on three criteria. First, they cover the main types of log anomaly detection methods, including traditional feature-based methods, sequence-based methods, Transformer-based methods, and semantic-enhanced methods. Second, they are widely used in prior studies and are suitable for direct comparison with our sequence-level detection setting. Third, their source code or clear implementation details are available, which allows us to reproduce them under the same datasets, windowing strategy, and evaluation metrics. Methods with different task settings, detection granularity, or unavailable implementation details are not included in the main comparison to ensure a fair and reproducible evaluation. The experimental results for all baselines were obtained by reproducing the open-source code from their original papers, all within our identical experimental environment. For each baseline model, we adopted the optimal parameter settings reported in their respective papers for training and evaluation.

- Principal Component Analysis (PCA) [26]: PCA detects anomalies by projecting a log sequence count matrix onto a lower-dimensional space and identifying outliers based on the projection length in the anomaly space.
- Support Vector Machines (SVM) [3]: SVM learns a hyperplane to distinguish between normal and anomalous log sequences in a high-dimensional space.
- DeepLog [6]: DeepLog uses LSTM to model system logs as natural language sequences, learning normal patterns and detecting anomalies when log patterns deviate from the model.
- LogBERT [9]: LogBERT combines a masked language model with a hypersphere volume minimization task to learn log event representations, and detects anomalies by checking if a sequence embedding lies outside of a learned hypersphere.
- LogAnomaly [7]: LogAnomaly combines log sequence features and statistical features, utilizing an LSTM-based deep learning model for anomaly sequence detection.
- LogCAE [27]: LogCAE combines active learning and contrastive learning to enhance log anomaly detection by efficiently incorporating limited human labels to optimize log representation in the feature space.
- LogFormer [11]: LogFormer improves the Attention module by supplementing it with parameter semantic information and employs a Transformer-based approach to perform sequence anomaly detection.

Implementation Details. In the experiment, LogMUSE uses a two-layer Transformer encoder, with 12 attention heads in each layer. The attention scales are set to $\{N, N/2, N/4\}$, where N represents the length of the divided log sequence. The Feed-Forward Network (FFN) layer has a size of 3072. During training, the Adam optimizer is used, with the learning rate controlled by OneCycleLR. The training process is controlled by early stopping, where training is terminated if no improvement is observed over 5 consecutive epochs. The model that performs best on the validation set is saved and used for testing.

All experiments are conducted in a Linux environment (kernel version 3.10.0) with Python 3.8.0 and PyTorch 2.4.1. The hardware platform consists of dual-socket Intel(R) Xeon(R) Platinum 8180 CPUs (112 cores, 2.50GHz), 384 GB RAM, and NVIDIA Quadro RTX 5000 GPUs (32 GB memory). The batch size is set to 64, with an initial learning rate of 1e-4.

Evaluation Metrics. To evaluate the effectiveness of LogMUSE in log anomaly detection, we use three evaluation metrics: Precision, Recall, and F1-score. These metrics are defined as follows:

- $Precision = \frac{TP}{TP+FP}$: It represents the proportion of true anomalous sequences among all sequences identified as anomalous. Higher precision indicates a lower false positive rate for anomalous sequences.
- $Recall = \frac{TP}{TP+FN}$: It represents the proportion of correctly identified anomalous sequences among all actual anomalous sequences. Higher recall indicates a lower false negative rate for anomalous sequences.
- $F1 - score = \frac{2 \times Precision \times Recall}{Precision + Recall}$: It represents the harmonic mean of Precision and Recall.

In these definitions, TP (True Positive) refers to the number of sequences correctly identified as anomalous, FP (False Positive) refers to the number of sequences incorrectly identified

TABLE II: Model comparison on BGL, Thunderbird and Spirit

Method	BGL			Thunderbird			Spirit		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
PCA	9.08%	98.23%	16.61%	37.35%	100.00%	54.39%	55.21%	98.32%	70.71%
SVM	10.60%	52.24%	17.63%	18.89%	79.11%	30.50%	56.62%	97.31%	71.59%
DeepLog	89.31%	76.52%	82.42%	80.72%	95.42%	87.46%	39.17%	96.32%	55.69%
LogAnomaly	82.72%	90.00%	86.21%	82.73%	97.61%	89.56%	48.62%	96.13%	64.58%
LogBERT	90.10%	90.69%	90.40%	91.75%	94.43%	93.07%	91.02%	97.22%	94.02%
LogCAE	91.12%	67.76%	77.72%	96.12%	82.12%	88.57%	96.06%	92.12%	94.04%
LogFormer	97.44%	97.31%	97.37%	98.73%	87.66%	92.87%	98.61%	99.40%	99.01%
LogMUSE	99.73%	97.54%	**98.62%**	94.33%	94.31%	**94.32%**	99.99%	99.60%	**99.20%**

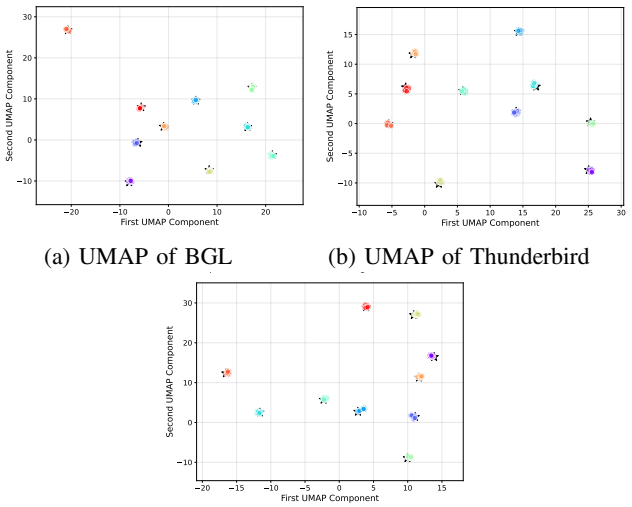
as anomalous, and FN (False Negative) refers to the number of anomalous sequences that were not correctly identified. Accuracy is not used to evaluate the effect of the model because the distribution of anomaly and normal is extremely unbalanced in the log sequence.

B. Evaluation of the Overall Performance

To demonstrate the effectiveness of LogMUSE, we compared it with other state-of-the-art log anomaly detection methods on the BGL, Thunderbird and Spirit datasets, with the results shown in Table II. All test results are averaged over 20 repeated trials. The results reveal that traditional machine learning methods, such as PCA and SVM, perform poorly in log anomaly detection tasks. While these methods achieve high precision or recall, they come with high false positives or false negatives, resulting in a low F1-score. This is primarily because these methods rely solely on count-based features and fail to capture key dependencies such as temporal relations, making it difficult to distinguish between normal and anomalous sequences.

In contrast, sequence-based deep learning methods, such as DeepLog and LogAnomaly, model sequential dependencies using LSTM and significantly outperform traditional machine learning methods. This highlights the importance of sequence dependencies in log anomaly detection, with deep learning methods effectively capturing log sequence patterns. However, LogBERT and LogFormer, which are based on Transformer models, differ in feature modeling. LogBERT uses only the log template indices as input, while LogFormer learns the inherent semantic features of the templates themselves. Despite LogBERT employing advanced sequence modeling techniques, its performance is comparable to that of DeepLog and LogAnomaly on the Thunderbird dataset, indicating that reliance on template indices alone fails to capture some anomaly patterns effectively. LogCAE attempts to enhance the separability of log sequences through a two-stage refinement process; however, its reliance on a conventional autoencoder for shallow encoding limits its semantic modeling capacity. As a result, it tends to memorize normal sequence patterns at a global level rather than effectively adapting to diverse and evolving anomaly scenarios.

In comparison, LogFormer enhances template-to-template dependencies by learning template semantics, leading to improved detection accuracy. Moreover, its supervised learning approach allows for better differentiation between anomalous and normal log sequences. Finally, LogMUSE achieves an F1-score of 98.62%, 94.32% and 99.20% on the BGL, Thunderbird and Spirit datasets, outperforming the current baseline methods. These results indicate that LogMUSE, through multi-scale modeling and parameter enhancement, effectively captures more sequence-level differences, resulting in superior anomaly detection performance.



(a) UMAP of BGL (b) UMAP of Thunderbird

(c) UMAP of Spirit

Fig. 5: BGL, Thunderbird and Spirit embed UMAP visualizations using enhanced templates

We also report ROC-AUC as a supplementary reference metric. LogMUSE achieves ROC-AUC values of 99.24%, 99.46%, and 100.00% on BGL, Thunderbird, and Spirit, respectively, which further supports its ability to distinguish anomalous sequences from normal sequences under different decision thresholds.

C. The Effectiveness of Log Semantic Embedding

To demonstrate that the semantic representation with parameter enhancement still effectively retains the original semantic information, while exhibiting some degree of variation based on the parameters, we apply Uniform Manifold Approximation and Projection (UMAP) for dimensionality reduction and visualization of both the original template semantics and the parameter-enhanced semantics. The results, as shown in Figure 5, indicate that all the newly generated enhanced templates

TABLE III: Comparison with different semantic embedding methods

Dataset	Method	Precision	Recall	F1-score
BGL	Log Key Index	98.12%	92.31%	95.13%
	Log Key BERT	97.81%	**97.74%**	97.77%
	Log Key + Parameter BERT	**99.73%**	97.54%	**98.62%**
Thunderbird	Log Key Index	90.16%	90.32%	90.24%
	Log Key BERT	93.21%	90.12%	91.64%
	Log Key + Parameter BERT	**94.33%**	**94.31%**	**94.32%**
Spirit	Log Key Index	96.11%	96.31%	96.21%
	Log Key BERT	97.21%	96.12%	96.66%
	Log Key + Parameter BERT	**99.99%**	**99.60%**	**99.20%**

cluster closely around their original templates, with a spread corresponding to the variations in parameters. Moreover, there is still a clear distinction between the enhanced vectors of different templates. This suggests that our method is able to effectively preserve the original semantics while incorporating parameter-enhanced features.

We further evaluated the impact of different embedding methods on the model's performance. LogMUSE was compared with two variations: *Log Key Index*, which embeds only the index information of log keys as semantic representations, and *Log Key BERT*, which uses sentence-BERT to embed log templates without considering parameter information. Our proposed method, *Log Key + Parameters BERT*, incorporates both log templates and parameter embeddings. The experimental results are shown in Table III.

The results indicate that when log semantics are not considered, the model can only capture the sequential relationships of log templates, failing to accurately represent the meaning of log messages. This limitation leads to significant false positives and false negatives. Incorporating log template semantics significantly improves model performance. Furthermore, integrating parameter information achieves the best results, demonstrating the critical role of parameters in understanding log semantics and improving anomaly detection accuracy.

D. The Effectiveness of Multi-Scale Selection

To evaluate the effect of multi-scale modeling, we test LogMUSE with different scale configurations, both individually and in combination. Following the dyadic scale design in Section III.C, we use three representative scales: N, $N/2$, and $N/4$, corresponding to global, medium-range, and relatively local dependencies, respectively. This setting covers different temporal ranges while avoiding redundant computation.

Table IV presents the impact of features at different scales on the model's detection performance across the BGL, Thunderbird and Spirit datasets. The results delineated in the table elucidate that the selection of varying scales exerts a differential influence on the efficacy of the model.

The experimental results on the BGL dataset reveal a consistent pattern in the impact of different scale choices on model performance. When selecting the global scale N, the model achieves high precision and F1-score, but the recall is slightly lower. This indicates that when the model focuses only on the entire sequence, it is effective in identifying most of the normal logs but may miss some localized anomaly patterns. As the scale is adjusted, using the $N/2$ scale results in a slight improvement in recall, while precision and F1-score remain largely unchanged, suggesting that the model can capture anomaly patterns more effectively at this scale without sacrificing overall precision. However, when the $N/4$ scale is used, precision decreases significantly, while recall remains nearly unchanged. This implies that when the scale is too small, the model can capture local anomalies but loses important global context information. This result also indicates that most anomalies occur within localized sequences, and thus, focusing on features at multiple scales is crucial for distinguishing between normal and anomalous sequences. By combining multiple scale selections, the model achieves better detection performance. Although the $N/4$ scale negatively impacts the detection of normal sequences, the combination of N and $N/2$ results in the highest F1-score and precision, despite a slight decrease in recall compared to using a single scale. This outcome further demonstrates that the inclusion of multi-scale features allows the model to comprehensively capture anomaly patterns across different scales, enhancing its stability and robustness.

A similar trend is observed in the Thunderbird dataset, where the model's performance stabilizes as the diversity of scale selections increases. The combination of multiple scales further enhances the F1-score, achieving the highest value when combining all scales N, $N/2$, and $N/4$ with an improvement of over 1% compared to using only the global scale. The Spirit dataset exhibits the same trend, where the integration of all three scales achieves the best performance, with an F1-score of 99.20%. These results highlight the effectiveness of multi-scale feature integration. Analyzing the dataset differences, Thunderbird and Spirit contain a greater number of log keys and log entries compared to BGL, with more dispersed anomalous entries. Consequently, smaller scales are more effective at capturing localized anomaly patterns in Thunderbird and Spirit.

Despite the differences in results across datasets, the benefits of multi-scale features remain consistent. The model demonstrates improved accuracy and robustness in anomaly detection across diverse datasets. By incorporating multi-scale features, the model captures both local and global characteristics, enabling better adaptation to varying data structures and anomaly patterns.

E. Evaluation of Different Model Parameters

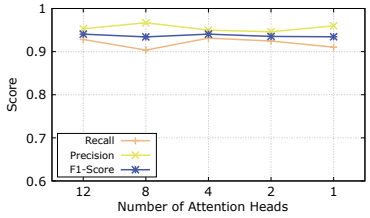
The number of attention heads and the size of the feedforward neural network are two critical parameters that influence model performance. To evaluate the impact of these parameters on log anomaly detection, we conducted performance tests by selecting different parameter combinations.

Figure 6a, 6b, and 6c illustrate the relationship between the number of attention heads and model performance. From the results, it can be observed that in the Thunderbird dataset (as shown in Figure 6a), F1-scores remain relatively stable,

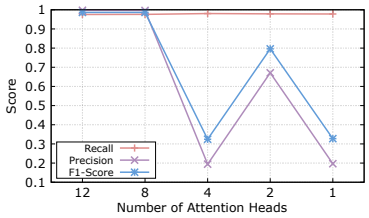
TABLE IV: Anomaly detection results for log sequences at different scales on BGL, Thunderbird and Spirit

Scales	BGL			Thunderbird			Spirit		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
$\{N\}$	96.81%	97.74%	97.27%	96.63%	89.70%	93.03%	99.80%	98.00%	98.89%
$\{N/2\}$	99.37%	**97.83%**	98.59%	97.37%	88.53%	92.74%	99.39%	98.00%	98.69%
$\{N/4\}$	90.53%	**97.83%**	94.04%	97.88%	87.42%	92.35%	99.99%	99.39%	98.80%
$\{N, N/2\}$	**99.73%**	97.54%	**98.62%**	**97.47%**	89.51%	93.32%	98.21%	98.80%	98.51%
$\{N, N/2, N/4\}$	99.30%	97.57%	98.43%	94.33%	**94.31%**	**94.32%**	**99.99%**	**99.60%**	**99.20%**

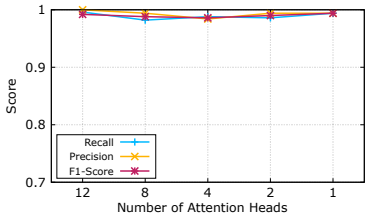
N is the length of the fixed window.



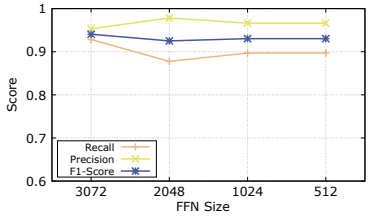
(a) Different attention heads on Thunderbird



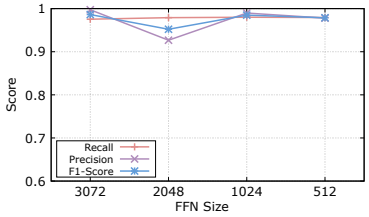
(b) Different attention heads on BGL



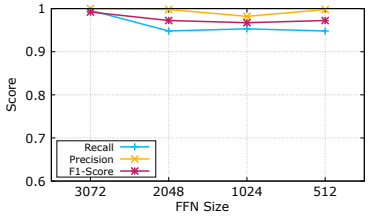
(c) Different attention heads on Spirit



(d) Different sizes of feed-forward networks on Thunderbird



(e) Different sizes of feed-forward networks on BGL



(f) Different sizes of feed-forward networks on Spirit

Fig. 6: Results of LogMUSE with different settings on BGL, Thunderbird and Spirit

ranging from 0.9341 to 0.9432, as the number of attention heads varies. Notably, when using fewer attention heads (e.g., 1 or 2), the model still achieves high precision but experiences a decline in recall. This suggests that simplified attention mechanisms may reduce false positives but struggle to capture anomalies effectively. The Spirit dataset (Figure 6c) shows a trend similar to Thunderbird, where performance remains relatively stable across different head configurations, suggesting that log diversity in Spirit can be effectively modeled even with fewer attention heads.

In contrast, the BGL dataset (as shown in Figure 6b) exhibits a markedly different trend. When using 12 or 8 attention heads, the model achieves an impressive F1-score of 0.9862, demonstrating exceptional performance. However, as the number of attention heads decreases, the model's performance significantly deteriorates, particularly in precision. This indicates that the model struggles to extract critical patterns in the sequence with fewer heads. The ability of attention heads to capture diverse dependency relationships is crucial for BGL, which contains a wider variety of error types. With fewer heads, the model fails to capture these dependencies effectively, resulting in degraded performance.

Thus, for anomaly detection tasks, it is recommended to select 8 to 12 attention heads, balancing the trade-off between computational efficiency and detection performance.

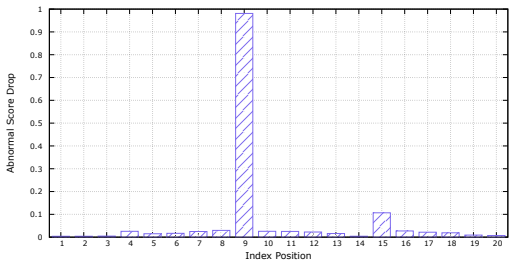
Figures 6d, 6e, and 6f present the relationship between FFN size and model performance. From the results, it is evident that larger FFN sizes generally yield better detection outcomes. For the Thunderbird dataset (as shown in Figure 6d), the F1-score reaches its highest value of 0.9405 when the FFN size is set to 3072, with balanced recall and precision. Similarly, in the BGL dataset (as shown in Figure 6e), an FFN size of 3072 achieves the best F1-score of 0.9862, indicating superior performance. And in Spirit dataset (shown in 6f), with only minor fluctuations across different FFN sizes and the best F1-score also observed at 3072. As the FFN size decreases, the F1-scores and recall rates drop slightly in these three datasets, but the precision remains relatively stable, demonstrating a degree of robustness. Overall, selecting an FFN size of 3072 provides optimal performance across both datasets, demonstrating the importance of sufficient FFN capacity in capturing complex and diverse log patterns for anomaly detection tasks.

F. A Simple Illustration of Detection

To better illustrate how LogMUSE performs anomaly detection, we present a case study using a randomly selected anomalous sequence from the Thunderbird dataset. For this sequence, we iteratively replaced each log entry with an all-zero vector and re-evaluated the anomaly score, in order to examine whether the model had effectively learned the contribution of each entry. As shown in Figure 7a, the 9-th

Index	Label	Content
1	-	ntpd[2272]: synchronized to 10.100.24.250, stratum 3
...	...	...
8	-	ib_sm.x[24904]: [ib_sm_sweep.c:411]: IB node 0x0005AD0000027550 port 21 is INIT state
9	N_CPU	kernel: Losing some ticks... checking if CPU frequency changed.
10	-	ntpd[2273]: synchronized to 10.100.28.250, stratum 3
...	...	...
20	-	smartd[1951]: Device: /dev/sda, Temperature changed 2 Celsius to 29 Celsius since last report

(a) An example of an abnormal log sequence



(b) The abnormal score drop after masking different log entries

Fig. 7: A simple illustration of detection

log entry reports a loss of ticks in clock interrupts, which may indicate potential system-level instability. This entry was labeled as type *N_CPU*, and its occurrence introduced an unexpected message that led to the sequence being classified as anomalous. Figure 7b depicts the decline in anomaly scores when different entries were masked. Notably, when the 9-th entry was replaced with an all-zero vector, the anomaly score dropped sharply and the sequence was classified as normal. This demonstrates that LogMUSE can accurately capture the crucial anomalous information within the sequence, effectively identifying the log entries that drive anomaly detection.

V. Discussion

A. Why does LogMUSE work?

Reasons for LogMUSE's superior performance over state-of-the-art methods can be summarized in two main aspects.

First, LogMUSE fully incorporates the semantic information of raw logs while introducing the impact of parameter information on anomaly detection. For log template embedding, we leverage the pre-trained Sentence-BERT model to capture the semantics of the template. Compared to the standard BERT model, Sentence-BERT, which employs the same tokenization approach, handles long sentences and complex semantics more effectively and directly provides sentence-level embeddings, making it better suited for semi-structured data like logs. Furthermore, we embed each log entry's parameters using a combination of character encoding and original position encoding. This embedding is then enhanced with cross-attention mechanisms to associate templates with parameter information effectively. This design increases the variability among logs with the same template, enabling the detection process to capture finer-grained differences and improve accuracy.

Second, LogMUSE introduces a multi-scale window-based feature extraction network that leverages a Transformer encoder to simultaneously focus on dependencies across different time steps. This approach fully captures both local and global features, which are then balanced and combined through weighted averaging, resulting in a more accurate and robust anomaly detection model. Since anomalies are often abrupt and rare, the highly correlated segments of the sequence are typically limited to a small portion. Unlike standard Transformer models, our method enhances these local features effectively, further improving anomaly detection performance.

B. Limitations and Future Work

Although LogMUSE has demonstrated strong performance in log anomaly detection, it is not without limitations, which also point to promising avenues for future research.

Limitations. First, our evaluation is conducted on three benchmark datasets, i.e., BGL, Thunderbird, and Spirit. Although these datasets are widely used, the selected multi-scale attention radii $\{N, N/2, N/4\}$ may not always be optimal for logs from different systems or with different temporal characteristics. This may limit the direct transferability of the model.

Second, LogMUSE assumes that log labels and log data are reliable. In practice, log annotations may contain noise, and adversarial environments may involve log tampering, fake event injection, or partial data loss. These factors may affect the robustness of the model. In addition, LogMUSE still relies on log parsing as a preprocessing step, so parsing errors may propagate to downstream detection.

Third, the current sequence partitioning strategy is based on fixed-size windows. This simple design may miss cross-task dependencies or long-term correlations in interleaved log streams. Moreover, the interpretability of LogMUSE remains limited, which may affect its use in operational scenarios where analysts need clear explanations.

Future work. In future work, we plan to improve LogMUSE in several directions. First, we will explore transfer learning and domain adaptation to enhance cross-system generalization. Second, we will study semi-supervised, unsupervised, and robust learning strategies to reduce the dependence on accurate labels and improve resistance to noisy or manipulated logs. Third, we will investigate dynamic log sequence partitioning to better capture task-specific and long-range dependencies. Finally, we will improve model interpretability and extend LogMUSE toward real-time anomaly detection in large-scale systems.

VI. Related Work

In this section, we review representative studies on log anomaly detection and organize them into five modeling paradigms. Table V summarizes the main characteristics of several classic methods in this field.

Early studies mainly relied on rule-based detection strategies [33], [34]. These methods identify abnormal events using expert knowledge, keyword matching, regular expressions, or predefined rules. Although they are simple and interpretable, their effectiveness depends heavily on manually designed rules. As modern systems become larger and more dynamic, such methods are often difficult to maintain and adapt. They

TABLE V: Comparison of Representative Log Anomaly Detection Methods Across Modeling Paradigms

Categories	Reference	Supervision	Embedding	Method	Window Division	Datasets
Machine Learning-based	Chen 2004 [3]	supervised	statistical feature count vector	SVM	global	BGL
	Xu 2009 [26]	supervised	statistical event count vector	PCA	global	HDFS and private Darkstar
Sequence-based	DeepLog [6]	unsupervised	event count vector	LSTM	global	HDFS and OpenStack
	LogAnomaly [7]	unsupervised	word2vec template semantic vector	LSTM	global	HDFS and BGL
	MLog [28]	supervised	Bert+Event IDF template semantic vector	Mogrifier LSTM+CNN	LSTM for global CNN for local	HDFS and BGL
Transformer-based	LogBERT [9]	unsupervised	BERT template semantic vector	Transformer	global	HDFS, BGL and Thunderbird
	NeuralLog [10]	unsupervised	BERT, raw log message semantic vector	Transformer	global	HDFS, BGL, Thunderbird, and Spirit
	LogFormer [11]	supervised	sentence-BERT template semantic vector with parameter	Transformer Adapter Tuning	global	HDFS, BGL and Thunderbird
LLM-based	LLMeLog [29]	supervised	LLM	Fine-tune BERT and Transformer	global	HDFS, BGL and Thunderbird
	LogLLM [30]	supervised	BERT	LLaMA	global	HDFS, BGL, Liberty and Thunderbird
Graph-based	HRGCN [31]	unsupervised	HetGNN	Deep SVDD	global	TraceLog and FlowGraph
	OCDiGCN [32]	unsupervised	graph node attributes	One-class GCN	global	HDFS, BGL, Thunderbird, Spirit, and Hadoop

also have limited ability to analyze anomalies from a broader contextual perspective.

Machine learning-based methods were later introduced to reduce the dependence on manual rules. Representative approaches, such as PCA [26], SVM [3], and Isolation Forest (IF) [35], detect anomalies by extracting statistical features from log sequences. These methods can improve detection efficiency in certain scenarios, but their performance is highly affected by log sequence segmentation and system changes, such as log format updates or the emergence of new log types. Moreover, statistical features usually provide limited information about temporal dependencies among log events, which restricts their applicability in complex system environments.

Since logs are generated in chronological order, sequence modeling has become a major direction for log anomaly detection. RNN- and LSTM-based methods [6]–[8], [28], [36]–[42] learn normal execution patterns from ordered log sequences and detect anomalies when these patterns are violated. For example, DeepLog [6] converts raw logs into template sequences and uses LSTM to model normal execution workflows. LogAnomaly [7] considers both template frequency and sequential dependencies, while LogRobust [8] combines TF-IDF-based template representation with BiLSTM for supervised anomaly detection. More recent methods, such as MLog [28] and LogESP [42], further improve local-global feature modeling and entry-level semantic representation. Despite these advances, sequence-based methods remain sensitive to unseen or evolving log patterns. They also tend to underuse the semantic information contained in log templates and parameters. In addition, their recurrent structures make it difficult to capture long-range dependencies efficiently.

Transformer-based models provide a more flexible way to capture log dependencies and semantic patterns. Compared with RNN- and LSTM-based methods, Transformer-based approaches [9]–[11], [19]–[21], [43]–[45] can model long-range dependencies without strictly relying on sequential recurrence. LogBERT [9] applies self-supervised Transformer learning to log sequences by treating log events in a way similar to natural language tokens. LogFormer [11] further improves log representation through a two-stage Transformer training strategy and adapts the model to different datasets with limited fine-tuning. Nevertheless, most existing Transformer-based methods still model log sequences at a single scale. As a result, they may focus mainly on global sequence patterns and fail to capture localized anomalies or temporal dependencies at different granularities. Recent studies in text modeling have shown that multi-scale attention can capture both local and global semantics [46]. Motivated by this idea, we design a multi-scale Transformer framework tailored to the heterogeneous temporal patterns of log anomaly detection.

The use of large language models (LLMs) for log anomaly detection has also attracted attention [29], [47], [48]. These methods adapt general-purpose pretrained language models to log analysis tasks and use their language understanding ability to improve log representation. For instance, LLMeLog [29] enhances log event representations with LLMs and combines fine-tuned BERT with a Transformer model for anomaly detection. LogLLM [30] extracts semantic vectors from log messages using BERT and employs a Transformer decoder for classification, with a projector used to align the representation spaces. Although these methods show the potential of LLMs in log analysis, most of them still transfer existing language models directly to logs. They provide limited customization for the specific structures and semantics of log data, and the development of log-specific large models remains an important direction for future research.

Another line of work converts log data into graph structures for anomaly detection. Graph-based methods [31], [32], [49],

[50] model relationships among logs, entities, and system components, which can improve the interpretability of detection results. For example, OCDiGCN [32] constructs heterogeneous graphs from parsed logs and applies graph neural networks for anomaly detection. Lograph [50] mines semantic associations in logs, builds a Log-Entity Graph, and uses a heterogeneous graph attention network to detect anomalous patterns. However, graph construction may introduce additional computational cost and can lose part of the sequential information in logs. These limitations make efficient online detection more difficult in large-scale systems.

VII. Conclusion

In this paper, we propose LogMUSE, a novel log anomaly detection method based on multi-scale semantic representation. LogMUSE effectively addresses the limitations of existing anomaly detection techniques, which often fail to capture the true execution semantics and struggle with balancing global and local dependencies. By extracting both template and parameter information through log parsing, LogMUSE leverages a pre-trained BERT model for template semantic embedding, further enhancing the representation of log entries via cross-attention mechanisms to capture diverse parameter features within the same template. Moreover, the multi-scale Transformer model enables the detection of anomalies at different scales, allowing for simultaneous focus on both global and local anomaly patterns in fixed-length log sequences. Extensive experiments on real-world benchmark datasets, including BGL, Thunderbird, and Spirit, demonstrate that LogMUSE significantly outperforms existing methods in terms of anomaly detection performance. Specifically, LogMUSE achieves F1-scores of 98.62%, 94.32%, and 99.20% on BGL, Thunderbird, and Spirit, respectively, demonstrating consistent improvements over representative baseline methods. Furthermore, the consistent performance across different datasets indicates its potential applicability to diverse real-world system scenarios.

References

[1] M. Chen, "A big data log system for millions of servers," Presentation at ArchSummit Shenzhen, July 2019. [Online]. Available: https://archsummit.infoq.cn/2019/shenzhen/presentation/1494

[2] Spacelift, "How much data is generated every day?" 2023. [Online]. Available: https://spacelift.io/blog/how-much-data-is-generated-every-day

[3] M. Chen, A. X. Zheng, J. Lloyd, M. I. Jordan, and E. Brewer, "Failure diagnosis using decision trees," in *International Conference on Autonomic Computing, 2004. Proceedings.* IEEE, 2004, pp. 36–43.

[4] P. Bodik, M. Goldszmidt, A. Fox, D. B. Woodard, and H. Andersen, "Fingerprinting the datacenter: automated classification of performance crises," in *Proceedings of the 5th European conference on Computer systems*, 2010, pp. 111–124.

[5] J.-G. Lou, Q. Fu, S. Yang, Y. Xu, and J. Li, "Mining invariants from console logs for system problem detection," in *2010 USENIX Annual Technical Conference (USENIX ATC 10)*, 2010.

[6] M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 1285–1298.

[7] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun *et al.*, "Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs." in *IJCAI*, vol. 19, no. 7, 2019, pp. 4739–4745.

[8] X. Zhang, Y. Xu, Q. Lin, B. Qiao, H. Zhang, Y. Dang, C. Xie, X. Yang, Q. Cheng, Z. Li *et al.*, "Robust log-based anomaly detection on unstable log data," in *Proceedings of the 2019 27th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*, 2019, pp. 807–817.

[9] H. Guo, S. Yuan, and X. Wu, "Logbert: Log anomaly detection via bert," in *2021 international joint conference on neural networks (IJCNN)*. IEEE, 2021, pp. 1–8.

[10] V.-H. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2021, pp. 492–504.

[11] H. Guo, J. Yang, J. Liu, J. Bai, B. Wang, Z. Li, T. Zheng, B. Zhang, J. Peng, and Q. Tian, "Logformer: A pre-train and tuning pipeline for log anomaly detection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 1, 2024, pp. 135–143.

[12] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *2017 IEEE international conference on web services (ICWS)*. IEEE, 2017, pp. 33–40.

[13] M. Du and F. Li, "Spell: Streaming parsing of system event logs," in *2016 IEEE 16th International Conference on Data Mining (ICDM)*. IEEE, 2016, pp. 859–864.

[14] S. Messaoudi, A. Panichella, D. Bianculli, L. Briand, and R. Sasnauskas, "A search-based approach for accurate identification of log message formats," in *Proceedings of the 26th Conference on Program Comprehension*, 2018, pp. 167–177.

[15] A. A. Makanju, A. N. Zincir-Heywood, and E. E. Milios, "Clustering event logs using iterative partitioning," in *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2009, pp. 1255–1264.

[16] Z. Ma, A. R. Chen, D. J. Kim, T.-H. Chen, and S. Wang, "Llmparser: An exploratory study on using large language models for log parsing," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[17] N. Reimers, "Sentence-bert: Sentence embeddings using siamese bert-networks," *arXiv preprint arXiv:1908.10084*, 2019.

[18] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.

[19] S. Huang, Y. Liu, C. Fung, R. He, Y. Zhao, H. Yang, and Z. Luan, "Hitanomaly: Hierarchical transformers for anomaly detection in system log," *IEEE transactions on network and service management*, vol. 17, no. 4, pp. 2064–2076, 2020.

[20] C. Almodovar, F. Sabrina, S. Karimi, and S. Azad, "Logfit: Log anomaly detection using fine-tuned language models," *IEEE Transactions on Network and Service Management*, 2024.

[21] T. Xiao, Z. Quan, Z.-J. Wang, Y. Le, Y. Du, X. Liao, K. Li, and K. Li, "Loader: A log anomaly detector based on transformer," *IEEE Transactions on Services Computing*, vol. 16, no. 5, pp. 3479–3492, 2023.

[22] S. Mallat, *A wavelet tour of signal processing*. Elsevier, 1999.

[23] S. He, J. Zhu, P. He, and M. R. Lyu, "Loghub: A large collection of system log datasets towards automated log analytics," *arXiv e-prints*, pp. arXiv–2008, 2020.

[24] A. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *37th annual IEEE/IFIP international conference on dependable systems and networks (DSN'07)*. IEEE, 2007, pp. 575–584.

[25] L. Yang, J. Chen, Z. Wang, W. Wang, J. Jiang, X. Dong, and W. Zhang, "Semi-supervised log-based anomaly detection via probabilistic label estimation," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1448–1460.

[26] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, 2009, pp. 117–132.

[27] P. Xiao, T. Jia, C. Duan, H. Cai, Y. Li, and G. Huang, "Logcae: An approach for log-based anomaly detection with active learning and contrastive learning," in *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2024, pp. 144–155.

[28] Y. Fu, K. Liang, and J. Xu, "Mlog: Mogrifier lstm-based log anomaly detection approach using semantic representation," *IEEE Transactions on Services Computing*, vol. 16, no. 5, pp. 3537–3549, 2023.

[29] M. He, T. Jia, C. Duan, H. Cai, Y. Li, and G. Huang, "Llmelog: An approach for anomaly detection based on llm-enriched log events," in *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2024, pp. 132–143.

[30] W. Guan, J. Cao, S. Qian, J. Gao, and C. Ouyang, "Logllm: Log-based anomaly detection using large language models," *arXiv preprint arXiv:2411.08561*, 2024.
[31] J. Li, G. Pang, L. Chen, and M.-R. Namazi-Rad, "Hrgcn: Heterogeneous graph-level anomaly detection with hierarchical relation-augmented graph neural networks," in *2023 IEEE 10th International Conference on Data Science and Advanced Analytics (DSAA)*. IEEE, 2023, pp. 1–10.
[32] Z. Li, J. Shi, and M. Van Leeuwen, "Graph neural networks based log anomaly detection and explanation," in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, 2024, pp. 306–307.
[33] M. Cinque, D. Cotroneo, and A. Pecchia, "Event logs for the analysis of software failures: A rule-based approach," *IEEE Transactions on Software Engineering*, vol. 39, no. 6, pp. 806–821, 2012.
[34] T.-F. Yen, A. Oprea, K. Onarlioglu, T. Leetham, W. Robertson, A. Juels, and E. Kirda, "Beehive: Large-scale log analysis for detecting suspicious activity in enterprise networks," in *Proceedings of the 29th annual computer security applications conference*, 2013, pp. 199–208.
[35] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 eighth ieee international conference on data mining*. IEEE, 2008, pp. 413–422.
[36] A. Brown, A. Tuor, B. Hutchinson, and N. Nichols, "Recurrent neural network attention mechanisms for interpretable system log anomaly detection," in *Proceedings of the first workshop on machine learning for computing systems*, 2018, pp. 1–8.
[37] Z. Wang, Z. Chen, J. Ni, H. Liu, H. Chen, and J. Tang, "Multi-scale one-class recurrent neural networks for discrete event sequence anomaly detection," in *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, 2021, pp. 3726–3734.
[38] Y. Yuan, S. S. Adhatarao, M. Lin, Y. Yuan, Z. Liu, and X. Fu, "Ada: Adaptive deep log anomaly detector," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 2449–2458.
[39] R. Yang, D. Qu, Y. Gao, Y. Qian, and Y. Tang, "Nlsalog: An anomaly detection framework for log sequence in security management," *IEEE Access*, vol. 7, pp. 181 152–181 164, 2019.
[40] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "Swisslog: Robust and unified deep learning based log anomaly detection for diverse faults," in *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2020, pp. 92–103.
[41] X. Wang, L. Yang, D. Li, L. Ma, Y. He, J. Xiao, J. Liu, and Y. Yang, "Maddc: Multi-scale anomaly detection, diagnosis and correction for discrete event logs," in *Proceedings of the 38th Annual Computer Security Applications Conference*, 2022, pp. 769–784.
[42] Z. Ni, X. Di, X. Liu, L. Chang, J. Li, and Q. Tang, "Logesp: Enhancing log semantic representation with word position for anomaly detection," in *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*. IEEE, 2024, pp. 1917–1924.
[43] S. Huang, Y. Liu, C. Fung, H. Wang, H. Yang, and Z. Luan, "Improving log-based anomaly detection by pre-training hierarchical transformers," *IEEE Transactions on Computers*, vol. 72, no. 9, pp. 2656–2667, 2023.
[44] S. Tao, Y. Liu, W. Meng, Z. Ren, H. Yang, X. Chen, L. Zhang, Y. Xie, C. Su, X. Oiao *et al.*, "Biglog: Unsupervised large-scale pre-training for a unified log representation," in *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*. IEEE, 2023, pp. 1–11.
[45] K. Qiu, Y. Zhang, Y. Feng, and F. Chen, "Loganomex: An unsupervised log anomaly detection method based on electra-dp and gated bilinear neural networks," *Journal of Network and Systems Management*, vol. 33, no. 2, pp. 1–29, 2025.
[46] Q. Guo, X. Qiu, P. Liu, X. Xue, and Z. Zhang, "Multi-scale self-attention for text classification," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 05, 2020, pp. 7847–7854.
[47] J. Qi, S. Huang, Z. Luan, S. Yang, C. Fung, H. Yang, D. Qian, J. Shang, Z. Xiao, and Z. Wu, "Loggpt: Exploring chatgpt for log-based anomaly detection," in *2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*. IEEE, 2023, pp. 273–280.
[48] X. Han, S. Yuan, and M. Trabelsi, "Loggpt: Log anomaly detection via gpt," in *2023 IEEE International Conference on Big Data (BigData)*. IEEE, 2023, pp. 1117–1122.
[49] J. Zeng, Z. L. Chua, Y. Chen, K. Ji, Z. Liang, and J. Mao, "Watson: Abstracting behaviors from audit logs via aggregation of contextual semantics." in *NDSS*, 2021.
[50] G. Chu, J. Wang, Q. Qi, H. Sun, Z. Zhuang, B. He, Y. Jing, L. Zhang, and J. Liao, "Anomaly detection on interleaved log data with semantic association mining on log-entity graph," *IEEE Transactions on Software Engineering*, 2025.

Mengyao Liu received the MSc degree in the School of Cyber Science and Technique, Beihang University, Beijing, China, in 2023. She is currently pursuing the Ph.D. degree in the School of Cyber Science and Technique, Beihang University, Beijing, China. Her current research interests include malware classification, log analysis, and information security.

Tianbo Wang (Member, IEEE) received the Ph.D. degree in computer application from Beihang University, Beijing, China, in 2018. He is currently an Associate Professor at Beihang University. He has participated in several National Natural Science Foundations and other research projects. His research interests include network and information security, intrusion detection technology, and information countermeasures.

Yuan Zhao received the master's degree from Shandong Normal University, Jinan, Shandong, China, in 2021. She is currently pursuing the Ph.D. degree with Beihang University, Beijing, China, under the supervision of Prof. Chunhe Xia. Her research interests include information security and network traffic analysis.

Chunhe Xia received the Ph.D. degree in computer application from Beihang University, Beijing, China, in 2003. He is currently a Supervisor and a Professor with Beihang University and the Director of the Beijing Key Laboratory of Network Technology. He has participated in different major national research projects and has published more than 100 research papers in important international conferences and journals. His current research interests include network and information security, information countermeasures, cloud security, and network measurement.

Yingming Zeng received the B.S. degree in computer software and theory from Beijing Jiaotong University, Beijing, China, in 2006, and the M.S. degree in computer software and theory from the Second Chinese Academy of Aerospace Science and Technology, China, in 2009. He is currently a Senior Engineer with Beijing Institute of Computer Technology and Applications, Beijing, China. His research interests include network security protection, artificial intelligence security, network attack and defense confrontation, etc., and he has presided over or participated in the research of more than 20 scientific and technological innovation topics in the field of network security.

Yuan Tao received the Ph.d. degree in control theory and control engineering from the University of Science and Technology Beijing, China, in 2010. He is currently a Professor with Classified Security Protection, the Third Research Institute of Ministry of Public Security, Shanghai, China. His research interests include the field of cyberspace security, including classified security protection, critical information infrastructure, and blockchain security.